Post-Quantum Finality for Lux Quasar

Replacing BLS Aggregation with ML-DSA + Corona

Zach Kelling
Lux Industries Research

2026-04-13

Abstract

The current Quasar consensus certificate combines BLS12-381 aggregation with the Corona and ML-DSA lattice threshold signatures for "defense in depth." We show that BLS contributes no post-quantum security to the combined cert and can be removed without loss of PQ soundness. The simplified Quasar certificate uses only ML-DSA (FIPS 204, Module-LWE/MSIS) and Corona (a Module-LWE threshold signature, the Lux fork of Ringtail (Boschini et al., IACR ePrint 2024/1113; see Related Work): module dimensions $M = 8$, $N = 7$ over $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$, $q =$ `0x1000000004A01`, $\sigma_E = 6.108$, NIST Category 1) — two independent lattice instances, each post-quantum. The result is operationally simpler, smaller in the common case, and provably PQ-safe under either instance independently.

Naming. "Corona" is the Lux first-party fork of Ringtail (Boschini et al., IACR ePrint 2024/1113; see Related Work), the two-round Module-LWE threshold signature; Corona adds a permissionless leaderless DKG, verifiable resharing, activation cert, and rollback suitable for validator rotation (LP-4440 / `github.com/luxfi/corona`). "Pulsar" (LP-073 / `github.com/luxfi/pulsar`) is a *separate* Module-LWE threshold scheme, parameterised to be byte-equal to FIPS 204 ML-DSA-65; it is not a fork of Corona, nor is Corona a fork of it. Both are Module-LWE — their diversity is parameter regime and lifecycle, not lattice family. The soundness arguments below apply to either signing primitive.

This document specifies:

1. the reduced three-layer Quasar cert: per-block committee cert, per-epoch Pulsar pulse, per-era Z-chain ZK proof;
2. the security argument (forgery requires breaking both independent Module-LWE instances — ML-DSA's and Corona's);
3. concrete efficiency numbers measured on MacBook M3 / arm64;
4. why a fixed-size committee of $k = 128$ is scale-invariant from $N = 10^3$ to $N = 10^5$ validators.

1 Motivation

The classical BLS12-381 pairing-based aggregate signature is broken in polynomial time by a sufficiently large quantum computer (Shor's algorithm on the discrete log in $\mathbb{F}_{q^{12}}$). Including BLS in a "PQ-safe" finality cert therefore provides only a classical speed-up; it cannot contribute to the post-quantum security argument.

This leaves Quasar with three natural reasons to keep BLS:

1. **Speed**: classical verification of a BLS aggregate is ∼1 ms.

2. **Size**: a BLS aggregate is 48 bytes; 128 individual ML-DSA signatures are 128×2420 = 309,760 bytes.

3. **Hedge**: if all lattice assumptions break simultaneously, BLS still holds until a quantum attacker materialises.

Against these we weigh:

- **False sense of security**: operators reading "PQ cert" may assume the cert is PQ when in fact it is only PQ conditional on verifying the lattice component.
- **Complexity cost**: three independent crypto stacks (BLS pairing, plus two distinct Module-LWE instances — ML-DSA and Corona) to maintain, benchmark, and audit.
- **PQ-first deployments**: networks that explicitly reject any classical primitive (regulated finance, long-archival chains) cannot use the BLS-inclusive cert at all.

We conclude the hedge argument does not justify the complexity and drop BLS.

2 Reduced Quasar Certificate

2.1 Per-block committee certificate (hot path)

For each block at height h:

1. A committee $C_h \subset \mathcal{V}$ of size $k = 128$ is sampled from the validator set $\mathcal{V}$ (of size N) via a post-quantum VRF:
$$C_h = \mathsf{Sample}_{\mathsf{VRF}}(\mathsf{prev_hash} \,\|\, h; \mathsf{stake}) \,.$$
The VRF uses a lattice-based construction (or ML-DSA itself as a hash-based VRF, with unpredictability in the ROM).

2. Each $v_i \in C_h$ signs the block header B_h with its individual ML-DSA-65 key:
$$\sigma_i = \mathsf{ML\text{-}DSA.Sign}(sk_i, B_h).$$

3. The block carries $\mathsf{cert}_h = \big(C_h^{\mathsf{bitmap}}, (\sigma_i)_{i \in C_h^{\mathsf{signed}}}\big)$.

4. A block is considered *locally final* iff $|C_h^{\mathsf{signed}}| \geq \lceil 2k/3 \rceil$.

This cert is $\leq 128 \times 2420 = 309{,}760$ bytes and verifies in ~ 5 ms on commodity hardware (measured below).

2.2 Per-epoch Corona aggregate (warm path)

An *epoch* spans E blocks (target: $E = 128$, one hour at 30-second block times). At the end of each epoch, the committee that signed block h_{end} produces a Corona threshold signature over the epoch tail Merkle root:

$$\sigma_{\text{epoch}} = \mathsf{Corona.Combine}\Big(\big(\mathsf{Corona.Sign}_i(r_{\text{epoch}})\big)_{i \in C_{h_{\text{end}}}^{\mathsf{signed}}}\Big),$$

where r_{epoch} is the Merkle root over $\{B_h\}_{h \in \text{epoch}}$. σ_{epoch} is $\sim$ 5–10 kB regardless of epoch length.

Once an epoch aggregate is available, validators may prune the individual block certs inside the epoch; archival nodes keep them for auditability.

2.3 Per-era Z-chain ZK proof (cold path)

An *era* spans R epochs (target: $R = 168$, one week at hourly epochs). At the end of each era, a Groth16 or PLONK proof π_{era} is produced on the Z-chain proving:

$$\pi_{\text{era}} : \exists\ (C_h, \sigma_h)_{h \in \text{era}} \ \text{ s.t. } \ \bigwedge_h \mathsf{QuasarVerify}(B_h, C_h, \sigma_h) = 1.$$

π_{era} is ≤ 200 bytes and verifies in ~ 3 ms. Light clients sync by downloading era proofs, not block certs.

3 Scheme-agnostic certificate and the signature registry

The committee certificate of the previous section is written with ML-DSA-65 as the per-signer primitive. In the shipped engine the primitive is *not* baked into the consensus rule: the engine defines only the *quorum predicate* (a stake-weighted $\frac{2}{3}$ rule), and the node injects the per-signer signature scheme as a wire-tagged parameter. This decoupling is what makes the classical→PQ migration a configuration flip rather than a protocol change, and it is the object this section formalises.

Definition 1 (Signature-scheme registry)**.** *A one-byte identifier* $\mathsf{SigSchemeID} \in \{0, \ldots, 255\}$ *names the scheme that produced a signature slot of a Quasar certificate. The registry (`config/pq_mode.go`) is block-structured and open-ended:*

`0x00`	*none (BLS-only certs commit to no scheme)*
`0x10`	BLS12-381 *aggregate (classical, day-1)*
`0x20`	*Nasua / academic R-LWE threshold (BLAKE3 profile)*
`0x30`	Pulsar.R *production R-LWE threshold*
`0x41/42/43`	*raw* ML-DSA-44/65/87 *(FIPS 204, single-party)*
`0x51/52/53`	Pulsar-M-44/65/87 *(M-LWE threshold)*

`0x52` *(*Pulsar-M-65*) is the production PQ default. The* `0x40` *and* `0x50` *blocks are kept distinct on purpose: both verify under the* unmodified *FIPS 204* ML-DSA.Verify *relation, but the producing protocol differs (single-party vs. threshold), and the wire separates them so a receiver knows which kernel emitted the bytes. New entries claim the next free integer in their block and are pinned in a test vector before any code references them; a retired id is never reused.*

Definition 2 (Scheme-parametric quorum predicate)**.** *Let* $s = \mathsf{SigSchemeID}$ *and let* Verify_s *be its verification relation. A certificate* $\mathsf{cert} = (s, \mathsf{pos}, \{(\mathsf{id}_i, \sigma_i)\}_{i \in T})$ *for position* pos *(chain, height, round, canonical commitment* C*, set-root) is accepted iff*

$$\mathsf{Prism}_s(\mathsf{cert}) \ = \ \Big[\ \textstyle\sum_{i \in T} \mathrm{stk}(\mathsf{id}_i) \ \geq \ \tau S \ \Big] \ \wedge \ \bigwedge_{i \in T} \mathsf{Verify}_s\big(pk_{\mathsf{id}_i},\ \mu(\mathsf{pos}, s),\ \sigma_i\big) \ \wedge \ \Phi_{\text{set}},$$

where $\tau \geq 2/3$*,* Φ_{set} *asserts each* id_i *is a distinct, current member of the validator set committed by* pos*'s set-root, and* $\mu(\mathsf{pos}, s)$ *is the signed transcript of Definition 3. The* quorum rule *— the bracketed stake test and* Φ_{set} *— is independent of* s*; only the per-signer relation* Verify_s *and the transcript tag depend on it.*

Definition 3 (Scheme-bound transcript)**.** *The message each signer signs binds the scheme id into the domain- separated transcript:*

$$\mu(\mathsf{pos}, s) \ = \ \mathsf{H}\big(\texttt{"LUX/chain/vote/v2"} \,\|\, \mathsf{chainID} \,\|\, \mathsf{height} \,\|\, \mathsf{round} \,\|\, C \,\|\, \mathsf{parentC} \,\|\, \mathsf{setRoot} \,\|\, s \,\|\, \mathsf{hashSuite}\big),$$

with H *a domain-separated SHAKE256 sponge (FIPS 202). The scheme id* s *and the hash-suite id are covered bytes, not envelope metadata.*

Lemma 4 (Cross-suite confusion resistance)**.** *Let* $s \neq s'$*. An adversary who takes a certificate honestly produced under scheme* s *and re-labels it* s' *(or splices a scheme-*s' *signature into a scheme-*s *position) produces a certificate that fails* $\mathsf{Prism}_{s'}$ *except with probability at most the EUF-CMA advantage against the underlying scheme.*

Proof. $\mathsf{Prism}_{s'}$ recomputes the transcript as $\mu(\mathsf{pos}, s')$ and checks $\mathsf{Verify}_{s'}(pk_i, \mu(\mathsf{pos}, s'), \sigma_i)$. The honest signers signed $\mu(\mathsf{pos}, s)$, and s is a covered input to H, so $\mu(\mathsf{pos}, s') \neq \mu(\mathsf{pos}, s)$ except for a collision of H (negligible under FIPS 202 collision resistance). Thus σ_i is a signature on a *different* message than the one $\mathsf{Verify}_{s'}$ checks; producing a σ_i that verifies on $\mu(\mathsf{pos}, s')$ under pk_i is exactly an EUF-CMA forgery against scheme s'. The confusion attack therefore fails on *signature verification*, not merely on a byte-equality tag check — the defense is cryptographic, not syntactic. □

Proposition 5 (PQ migration is a profile flip)**.** *Fix a validator set and stake distribution. Let* $\mathsf{Prism}_s, \mathsf{Prism}_{s'}$ *be the predicates of Definition 2 under schemes* s, s' *(e.g.* $s =$ `0x10` *classical BLS,* $s' =$ `0x42` *raw* ML-DSA-65*, or* $s' =$ `0x52` Pulsar-M-65*). Then the safety analysis — the quorum-intersection argument of Theorem 7 and of the no-double-finalization theorem (*`quasar-cert-soundness.tex`*, §"Finality Admission") — is* invariant *under the substitution* $s \mapsto s'$*; only the per-signer forgery bound changes from* ϵ_s *to* $\epsilon_{s'}$*.*

Proof. Both theorems use only two facts about the per-signer scheme: (i) that Prism accepts iff signer stake $\geq \tau S$ over distinct current set members — the quorum rule, which Definition 2 makes independent of s; and (ii) that an honest validator's signature on one transcript cannot be forged, i.e. Verify_s is EUF-CMA with advantage ϵ_s. Fact (i) is unchanged by the substitution. Fact (ii) holds for every scheme in the registry (BLS under co-CDH; ML-DSA and Pulsar-M under MLWE/MSIS; the threshold variants under their SUF-CMA proofs), with the bound $\epsilon_{s'}$ substituted for ϵ_s. No other property of the primitive enters either proof. Hence enabling the strict-PQ profile — which pins the validator identity/finality scheme to ML-DSA-65/87 or Pulsar-M-65/87 and refuses classical schemes — is a re-parameterisation that preserves every safety invariant and strictly improves the quantum-forgery bound (from the Shor-breakable ϵ_{BLS} to the lattice $\epsilon_{\mathrm{MLWE,MSIS}}$). The flip is a *value*, not a new protocol. □

Remark 6 (Assumption diversity, honestly)**.** Pulsar *(R-LWE / M-LWE threshold) and* ML-DSA *(M-LWE, FIPS 204) both rest on* lattice *assumptions; a single sufficiently strong lattice cryptanalysis would dent both. The registry therefore also reserves the hash-based* SLH-DSA *(FIPS 205) family for the recovery and high-value roots (*`RecoverySchemeID` `0x05`–`0x07`*): a stateless hash-based signature survives a total break of lattice assumptions and is the genuine cross-family diversifier, not a second lattice leg. Two lattice legs buy defense-in-depth against implementation faults, not independent hardness.*

4 Security

4.1 Threat model

Let f be the Byzantine fraction of $\mathcal{V}$. Classical Byzantine thresholds require $f < 1/3$. Let the adversary $\mathcal{A}$ be any probabilistic polynomial time quantum machine (BQP).

4.2 Forgery reduction

Theorem 7 (PQ soundness of Quasar committee cert)**.** *Assume* ML-DSA-65 *is existentially unforgeable under the* $\mathsf{MLWE}_{q,k,\ell,\chi}$ *and* MSIS *assumptions, and that the VRF output is pseudorandom in the quantum random oracle model.*

Then an adversary $\mathcal{A}$ *that produces a valid* cert_h *for a block* B_h *not signed by* $\geq 2k/3$ *honest committee members breaks* ML-DSA *or predicts the VRF with non-negligible advantage.*

Proof sketch. By a standard hybrid argument:

H0: Real game. $\mathcal{A}$ forges cert_h containing $\geq 2k/3$ valid ML-DSA signatures over B_h from committee members.

H1: Replace VRF output with a uniform random sampling. Indistinguishable by VRF PQ-pseudorandomness.

H2: Committee is independent of $\mathcal{A}$'s choice. Probability of an unchallenged forgery on $\geq 2k/3$ honest keys is at most $\binom{k}{2k/3} \cdot \varepsilon_{\text{ML-DSA}}^{2k/3}$ where $\varepsilon_{\text{ML-DSA}}$ is the per-signature forgery advantage. For $k = 128$ and $\varepsilon_{\text{ML-DSA}} \leq 2^{-192}$ (Level III), this is $\leq 2^{-86} \cdot 2^{-192 \cdot 85} \approx 2^{-16 \cdot 10^3}$.

The adversary's overall advantage is bounded by $\mathsf{Adv}^{\mathsf{VRF}} + \binom{k}{2k/3}\varepsilon_{\text{ML-DSA}}^{2k/3}$, which is negligible under our assumptions. □

4.3 Independence of the two lattice instances

Both lattice legs rest on Module-LWE (with Module-SIS for binding), so this is *not* family-disjoint diversity — it is two independent *instances* of the same family. ML-DSA is secure under $\mathsf{MLWE}_{q,k,\ell,\chi}$ and MSIS at the FIPS 204 parameters ($q = 8\,380\,417$, ring degree 256, module rank $(k, \ell) = (6, 5)$). Corona is secure under MLWE and MSIS at its own (Ringtail-derived) parameters: a much larger 48-bit modulus $q =$ `0x1000000004A01` over the same ring degree 256, with module dimensions $M = 8$, $N = 7$ and $\sigma_E = 6.108$.

No reduction relating the two concrete instances is known, and the two parameter regimes, key formats, and codebases are disjoint, so a break that exploits one parameterisation or one implementation need not affect the other. This is implementation- and parameter-level defense in depth, not a hedge across distinct hardness families: a structural break against Module-LWE *as a family* would threaten both legs simultaneously. The only truly family-disjoint signature leg in the Lux stack is the hash-based Magnetar (SLH-DSA / FIPS 205); cross-family hedging requires adding that leg, not the second lattice instance.

Proposition 8 (Hedged PQ security within Module-LWE)**.** *Under the conjunction "the ML-DSA Module-LWE instance is hard* $\vee$ *the Corona Module-LWE instance is hard," the combined per-epoch certificate* $(\mathsf{cert}_h$ *for* $h \in$ *epoch*$, \sigma_{epoch})$ *is unforgeable.*

This gives us defense in depth against parameter- or implementation-specific breaks strictly within the post-quantum setting, without relying on any classical primitive; for cross-family assurance, compose the Magnetar (hash) leg as in the Polaris profile.

5 Why BLS contributes no PQ security

Lemma 9. *A BLS12-381 aggregate signature* σ_{BLS} *can be forged efficiently by a quantum adversary running Shor's algorithm on the discrete log problem in the extension field* $\mathbb{F}_{q^{12}}$ *underlying the pairing target group.*

Consequently, any certificate whose verification *requires* σ_{BLS} to be valid can be broken by breaking BLS alone. If the Quasar cert verification logic is $\mathsf{Verify}_{\text{BLS}}(\cdot) \wedge \mathsf{Verify}_{\text{PQ}}(\cdot)$, then forging the cert still requires breaking the PQ components, so BLS adds a verification cost without adding an attack barrier. If the verification is $\mathsf{Verify}_{\text{BLS}}(\cdot) \vee \mathsf{Verify}_{\text{PQ}}(\cdot)$, then forging the cert requires breaking *only* BLS, reducing PQ security to zero.

In practice Quasar uses the $\wedge$-form, so BLS is safe but superfluous. Since it does not strengthen the cert, we remove it.

6 Scale invariance of fixed-committee signing

6.1 Claim

For a fixed committee size k, the per-block cert generation and verification cost is $O(k)$, independent of the total validator count N. Only the VRF sampling is touched by N, and the VRF sampling cost is $O(\log N)$ with a weighted-reservoir or alias-method implementation.

6.2 Measured costs on MacBook M3 (10-core arm64)

Using `mldsa-bench` at ML-DSA-44:

N	k	Keygen	Sign (parallel)	Verify (parallel)	Cert size
10^3	128	14.8 ms	3.8 ms	4.4 ms	310 kB
10^4	128	14.8 ms	3.8 ms	4.4 ms	310 kB
10^5	128	14.8 ms	3.8 ms	4.4 ms	310 kB

(Keygen is amortized across validator lifetime and only involves the committee members for a given block; in practice keys are generated once at validator registration. Sign and verify scale linearly with k only.)

6.3 Disk and bandwidth

	Per block	Per epoch (128 blocks)	Per era (168 epochs)
Raw ML-DSA committee cert	310 kB	40 MB	6.7 GB
After Corona roll-up	—	5–10 kB	1.2 MB
After Z-chain ZK proof	—	—	200 bytes

An archival node retains all raw certs (6.7 GB/era). A full node keeps the current era's raw certs plus all past Corona epoch aggregates ($\sim$200 MB/year). A light client synchronises from Z-chain era proofs only ($\sim$10 kB/year).

7 Recommendation

1. Adopt the reduced Quasar cert (ML-DSA + Corona + Z-chain ZK). Remove BLS from Quasar cert generation and verification.

2. Keep BLS available as an *opt-in* classical speed-up for deployments that explicitly accept non-PQ security in exchange for sub-millisecond verification of a 96-byte aggregate. Gate via a chain-configuration flag, off by default.

3. Use $k = 128$ fixed committee size for all network sizes in the range $10^3 \leq N \leq 10^5$.
4. Use ML-DSA-65 (NIST Level III) for the individual committee signatures.
5. Require a PQ VRF for committee sampling. Either a lattice VRF (see Boneh–Dai 2024) or ML-DSA output used as a pseudorandom source in the QROM.
6. Re-anchor every epoch via a Corona threshold aggregate for space efficiency.
7. Re-anchor every era via a Z-chain Groth16 or PLONK proof for light-client succinctness.

8 Open questions

- **PQ VRF**: which construction best balances cost and PQ-security? Lattice VRFs are still an active research area.
- **Stake-weighted sampling with liveness**: how do we recover when $> 1/3$ of stake is offline in a specific epoch? Committee rotation helps but a formal liveness bound is needed.
- **Corona parameter selection**: parameters for $k = 128$ threshold Corona have not been published; we may need a sensitivity analysis before adopting.

9 Related work

- Celi, del Pino, Espitau, Niot, Prest. *Efficient Threshold ML-DSA.* USENIX Security 2026.
- Boschini, Kaviani, Lai, Malavolta, Takahashi, Tibouchi. *Ringtail: Practical Two-Round Threshold Signatures from LWE.* IEEE S&P 2025 (IACR ePrint 2024/1113). The construction Corona forks.
- Ethereum beacon chain attester committee design (BLS-based).
- Avalanche / Snowman probabilistic sampling family.